Design and Verification of an 8-bit 5-Stage Pipelined RISC Processor

Verilog HDL implementation with data forwarding, load-use stall and branch flush, verified and implemented in AMD Vivado 2026.1

Project Report

Student	Lê Kỳ Dũng
Student ID	SE190745

Tool	AMD Vivado 2026.1 (XSim, Synthesis, Implementation)
Target device	Xilinx Artix-7 `xc7a35tcpg236-1`
Date	October 22, 2026

Abstract

This report presents the design, verification and FPGA implementation of a small 8-bit RISC processor organised as a classic five-stage pipeline (Instruction Fetch, Instruction Decode, Execute, Memory access and Write Back). The processor has eight 8-bit general-purpose registers, a 16-bit fixed-length instruction format, 16 instructions and separate instruction and data memories (Harvard architecture). All pipeline hazards are handled in hardware: read-after-write data hazards are removed by forwarding from the EX/MEM and MEM/WB pipeline registers together with a write-through register file, a load followed by a dependent instruction causes a single stall cycle, and taken branches and jumps flush the two wrong-path instructions. The design is written in synthesisable Verilog-2001 as seven modules and is verified with a self-checking testbench in the Vivado simulator. The 34-instruction test program completes in 51 clock cycles (CPI = 1.50) and all nine architectural checks pass. After implementation on an Artix-7 `xc7a35t` device, the core uses 240 LUTs and 186 flip-flops and meets the 100 MHz timing constraint with a worst negative slack of +1.747 ns, corresponding to a maximum frequency of about 121 MHz.

Contents

1 Introduction **4**

1.1 Motivation 4

1.2 Objectives 4

1.3 Scope 4

1.4 Report organisation 4

2 Background **4**

2.1 The five-stage pipeline 4

2.2 Pipeline hazards 5

3 Instruction Set Architecture **5**

3.1 Programmer-visible state 5

3.2 Instruction formats 6

3.3 Instruction set 6

3.4 Encoding example 6

4 Microarchitecture **7**

4.1 Top-level datapath 7

4.2 Module hierarchy 7

4.3 Instruction fetch (IF) 8

4.4 Instruction decode (ID) 8

4.5 Execute (EX) 8

4.6 Memory access (MEM) and write back (WB) 9

4.7 Pipeline registers 9

5 Hazard Handling **9**

5.1 Data hazards: forwarding 9

5.2 Load-use hazard: one-cycle stall . . . 10
5.3 Control hazards: flush . . . 10
5.4 Halt . . . 10

6 Verification **11**
6.1 Testbench . . . 11
6.2 Test program . . . 11
6.3 Simulation results . . . 11
6.4 Cycle-by-cycle trace . . . 12
6.5 Waveform . . . 14
6.6 Performance analysis . . . 14
6.7 Additional verification . . . 15

7 Synthesis and Implementation Results **15**
7.1 Project settings . . . 15
7.2 Resource utilisation . . . 15
7.3 Timing . . . 16
7.4 Power . . . 17
7.5 Methodology warnings . . . 17

8 Discussion **17**
8.1 Design decisions . . . 17
8.2 Limitations . . . 17
8.3 Future work . . . 17

9 Conclusion **18**

A Source Code **19**
A.1 `design_sources/alu.v` . . . 19
A.2 `design_sources/regfile.v` . . . 19
A.3 `design_sources/imem.v` . . . 20
A.4 `design_sources/dmem.v` . . . 21
A.5 `design_sources/control_unit.v` . . . 22
A.6 `design_sources/hazard_unit.v` . . . 23
A.7 `design_sources/risc8_5stage_core.v` . . . 24
A.8 `sim_sources/tb_risc8_5stage.v` . . . 29
A.9 `constraints/risc8.xdc` . . . 31

B Reproducing the Results in Vivado **31**

List of Figures

1 Ideal pipelined execution of three independent instructions 5
2 The four 16-bit instruction formats 6
3 Pipelined datapath of the RISC8 processor 7
4 Design hierarchy in Vivado: `risc8_5stage_core` is the top module with six sub-modules 8
5 Load-use hazard: `ADDI` waits one cycle in ID, then receives R5 from MEM/WB . . 10
6 Taken branch: the two wrong-path instructions are flushed (cycle 10, `flush=1`) . . 10
7 Simulation results and statistics printed by the testbench in the Vivado Tcl Console 12
8 Simulation trace, cycles 1–19 (forwarding, BZ flush and the first loop iterations) . 13
9 Simulation trace, cycles 19–50 (loop, JMP flush and load-use stall) 14
10 Behavioural simulation waveform in Vivado (0–526 ns) 14
11 Vivado project settings 15
12 Vivado Design Runs: synthesis (`synth_1`) and implementation (`impl_1`) completed 15
13 Vivado dashboard: utilisation after synthesis and placement, and timing summary 16

List of Tables

1 The five pipeline stages 5
2 Instruction set (16 instructions) 6
3 Verilog modules of the design 7
4 Main control signals generated in the ID stage 8
5 Pipeline register contents 9
6 Test program 11
7 Final architectural state 12
8 Resource utilisation on `xc7a35tcpg236-1` 16
9 Timing summary after routing (100 MHz constraint) 16

1 Introduction

1.1 Motivation

Pipelining is the most important technique used to increase the throughput of a processor. Instead of finishing one instruction completely before starting the next one, the work of an instruction is split into several stages, and different instructions occupy different stages at the same time. In the ideal case one instruction completes in every clock cycle, even though each instruction still needs several cycles to pass through the pipeline.

Pipelining also introduces *hazards*: situations in which the next instruction cannot execute correctly in the next cycle. Designing a pipeline therefore means designing not only the datapath, but also the logic that detects and resolves these hazards. A small processor is an ideal vehicle for studying these ideas, because every signal can be observed in simulation and the complete design fits into a few hundred lines of Verilog.

1.2 Objectives

The objectives of this project are:

1. Define a simple 8-bit instruction set architecture (ISA) with 16 instructions.
2. Design a five-stage pipelined datapath and control unit for this ISA in Verilog HDL.
3. Resolve all data and control hazards in hardware (forwarding, stalling and flushing).
4. Verify the design with a self-checking testbench and a test program that exercises every hazard case.
5. Synthesise and implement the design for a Xilinx Artix-7 FPGA and check that it meets a 100 MHz clock constraint.

1.3 Scope

The project covers RTL design, behavioural simulation, synthesis, placement and routing, and timing analysis in AMD Vivado 2026.1. Programming a physical FPGA board is outside the scope of this work; the constraint file therefore defines only the clock and does not assign package pins.

1.4 Report organisation

Section 2 reviews pipelining and hazards. Section 3 defines the instruction set. Section 4 describes the microarchitecture and each Verilog module. Section 5 explains how hazards are handled. Section 6 presents the verification method and simulation results. Section 7 reports the synthesis and implementation results. Section 8 discusses limitations and future work, and Section 9 concludes. The complete source code is given in Appendix A, and the steps to reproduce the results are in Appendix B.

2 Background

2.1 The five-stage pipeline

The processor follows the classic five-stage organisation popularised by the MIPS architecture. Table 1 lists the stages.

Table 1: The five pipeline stages

Stage	Name	Work performed
IF	Instruction Fetch	Read the instruction at address PC from instruction memory; PC ← PC+1.
ID	Instruction Decode	Decode the opcode, generate control signals, read the source registers.
EX	Execute	Perform the ALU operation, update the Z/C flags, decide branches and jumps.
MEM	Memory access	Read or write the data memory (LD/ST instructions only).
WB	Write Back	Write the result into the destination register.

Between two neighbouring stages there is a *pipeline register* (IF/ID, ID/EX, EX/MEM and MEM/WB). At every rising clock edge each pipeline register captures the outputs of the stage before it, so an instruction moves one stage forward per cycle. Figure 1 shows the ideal overlap of three independent instructions: from cycle 5 onwards one instruction completes in every cycle.

Cycle	1	2	3	4	5	6	7
Instr. 1	IF	ID	EX	MEM	WB		
Instr. 2		IF	ID	EX	MEM	WB	
Instr. 3			IF	ID	EX	MEM	WB

Figure 1: Ideal pipelined execution of three independent instructions

2.2 Pipeline hazards

A hazard prevents the next instruction from executing in its designated cycle. Three classes exist:

- **Structural hazards** occur when two instructions need the same hardware resource at the same time. They are avoided here by using separate instruction and data memories and a register file with two read ports and one write port.
- **Data hazards** occur when an instruction needs a result that an earlier instruction has not yet written back (read-after-write, RAW).
- **Control hazards** occur when the address of the next instruction is not known yet, because a branch or jump has not been resolved.

The standard solutions are *forwarding* (bypassing), which passes a result directly from a later stage back to the EX stage; *stalling*, which holds the younger instructions and inserts a bubble (a no-operation) into the pipeline; and *flushing*, which cancels instructions fetched from the wrong path.

3 Instruction Set Architecture

3.1 Programmer-visible state

- Eight 8-bit general-purpose registers `R0`–`R7`. `R0` is an ordinary register (it is not hard-wired to zero).
- An 8-bit program counter `PC`, addressing up to 256 instructions.

- Two condition flags: `Z` (zero) and `C` (carry/borrow).
- A 256 × 16-bit instruction memory and a 256 × 8-bit data memory (Harvard architecture).

3.2 Instruction formats

All instructions are 16 bits long. The upper four bits always hold the opcode. Four formats are used (Figure 2).

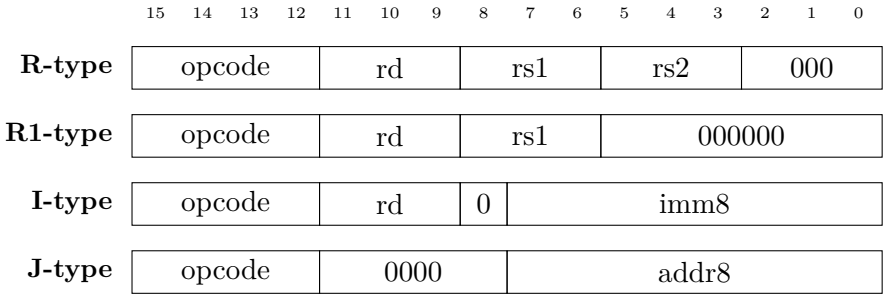


Figure 2: The four 16-bit instruction formats

3.3 Instruction set

Table 2 lists all 16 instructions. Arithmetic, logic and shift instructions update both flags; logic instructions keep the previous carry. `LDI` does not change the flags, which allows a constant to be loaded between a compare and the branch that tests it.

Table 2: Instruction set (16 instructions)

Opcode	Mnemonic	Format	Operation	Flags	Example
0	NOP	–	no operation	–	`0000`
1	ADD	R	rd ← rs1 + rs2	Z,C	`1488`
2	SUB	R	rd ← rs1 − rs2	Z,C	`2FB0`
3	AND	R	rd ← rs1 AND rs2	Z	`3650`
4	OR	R	rd ← rs1 OR rs2	Z	`4850`
5	XOR	R	rd ← rs1 XOR rs2	Z	`5CE0`
6	SHL	R1	rd ← rs1 ≪ 1, C ← bit 7	Z,C	`6D80`
7	SHR	R1	rd ← rs1 ≫ 1, C ← bit 0	Z,C	`76C0`
8	LDI	I	rd ← imm8	–	`823C`
9	ADDI	I	rd ← rd + imm8	Z,C	`92FF`
A	LD	R1	rd ← MEM[rs1]	–	`AB00`
B	ST	R1	MEM[rs1] ← rd	–	`B500`
C	JMP	J	PC ← addr8	–	`C014`
D	BZ	J	if Z = 1: PC ← addr8	–	`D00A`
E	BNZ	J	if Z = 0: PC ← addr8	–	`E00C`
F	HLT	–	stop the processor	–	`F000`

3.4 Encoding example

The first instruction of the test program is `0x823C`:

$$\texttt{0x823C} = \underbrace{1000}_{\text{opcode=8 (LDI)}}\ \underbrace{001}_{\text{rd=R1}}\ \underbrace{0}_{\text{unused}}\ \underbrace{0011\,1100}_{\text{imm8=0x3C}}$$

so it means `LDI R1, 0x3C`, i.e. load the constant `0x3C` into register `R1`. Similarly, `0x1488` = `0001 010 010 001 000` encodes `ADD R2, R2, R1`.

4 Microarchitecture

4.1 Top-level datapath

Figure 3 shows the datapath. The four shaded bars are the pipeline registers. Red arrows are the forwarding paths, and the dashed arrow is the branch target returned to the PC.

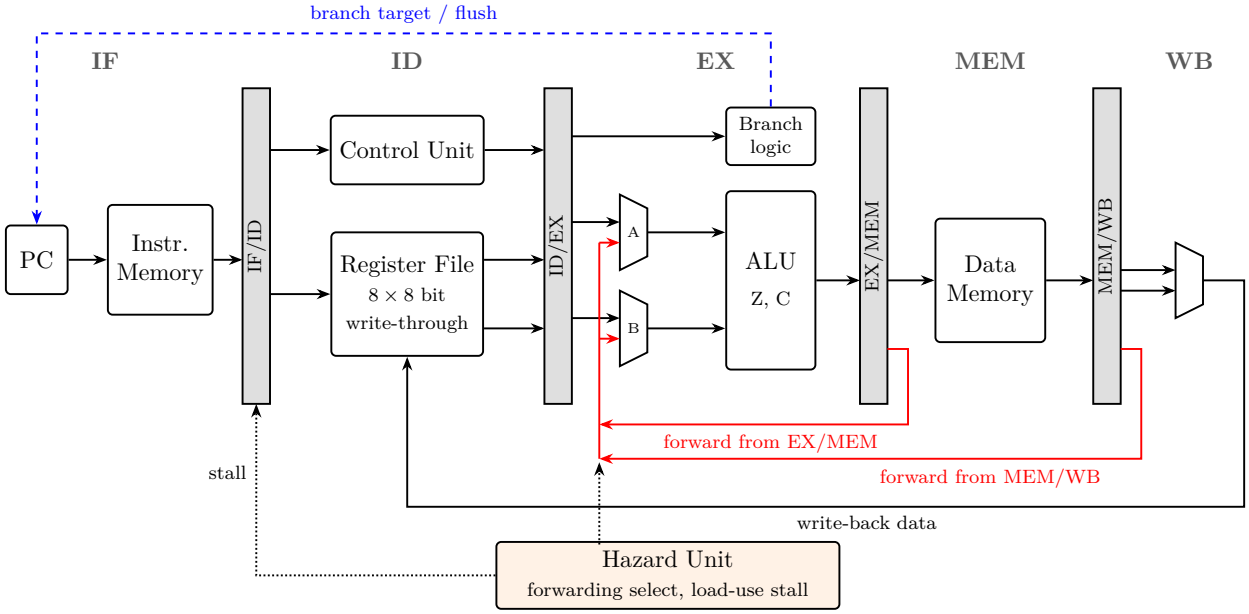


Figure 3: Pipelined datapath of the RISC8 processor

4.2 Module hierarchy

The design is split into seven Verilog modules (Table 3). The top module `risc8_5stage_core` contains the PC, the four pipeline registers, the flag register, the forwarding multiplexers and the branch logic, and instantiates the other six modules. Figure 4 shows the hierarchy in the Vivado *Sources* window.

Table 3: Verilog modules of the design

File	Instance	Stage	Function
`imem.v`	`u_imem`	IF	256 × 16 instruction ROM holding the test program, asynchronous read.
`control_unit.v`	`u_ctrl`	ID	Combinational decoder: opcode → 14 control signals.
`regfile.v`	`u_rf`	ID / WB	8 × 8 register file, 2 read ports, 1 write port, write-through bypass.
`alu.v`	`u_alu`	EX	8-bit ALU (ADD, SUB, AND, OR, XOR, SHL, SHR, PASS) with Z and C flags.
`dmem.v`	`u_dmem`	MEM	256 × 8 data RAM, asynchronous read, synchronous write.
`hazard_unit.v`	`u_hazard`	ID / EX	Forwarding selection and load-use stall detection.
`risc8_5stage_core.v`	(top)	all	PC, pipeline registers, flags, forwarding muxes, branch, flush and halt logic.

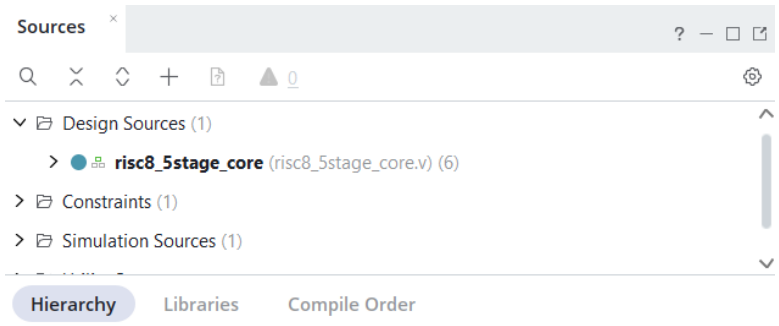


Figure 4: Design hierarchy in Vivado: `risc8_5stage_core` is the top module with six sub-modules

4.3 Instruction fetch (IF)

The instruction memory is a combinational ROM described with a `case` statement, so the program is part of the design and cannot be lost. The instruction at address PC is available in the same cycle. In the normal case the PC is incremented by one and the instruction is written into the IF/ID register together with a `valid` bit. The valid bit marks bubbles: a cleared valid bit turns the instruction into a no-operation, independently of its encoding.

4.4 Instruction decode (ID)

The control unit is a single combinational `always @(*)` block. All outputs are first assigned zero (equivalent to `NOP`), and each opcode then switches on only the signals it needs; this style guarantees that no latches are inferred. Table 4 summarises the most important control signals.

Table 4: Main control signals generated in the ID stage

Signal	Meaning
`alu_op[2:0]`	ALU operation: 0 ADD, 1 SUB, 2 AND, 3 OR, 4 XOR, 5 SHL, 6 SHR, 7 PASS-B.
`alu_src_imm`	Operand B of the ALU is the 8-bit immediate (LDI, ADDI).
`ra_is_rd`, `rb_is_rd`	Read port A/B reads `rd` instead of `rs1`/`rs2` (ADDI, ST).
`uses_a`, `uses_b`	The instruction really reads port A/B; used by the hazard unit to avoid false dependencies.
`reg_we`	Write the destination register in WB.
`mem_to_reg`	The result comes from data memory (LD).
`mem_we`	Write data memory in MEM (ST).
`flag_we`	Update the Z and C flags in EX.
`jump`, `br_z`, `br_nz`, `halt`	Control-flow instructions.

The register file has two asynchronous read ports and one synchronous write port. A *write-through bypass* returns the value being written in WB if the same register is read in ID in the same cycle. This removes the distance-3 data hazard without any extra hardware in the EX stage.

4.5 Execute (EX)

Two 3-input forwarding multiplexers select each ALU operand from the ID/EX register, the EX/MEM register or the MEM/WB register. A second multiplexer selects either operand B or the immediate. The ALU computes the result and the Z and C flags. ADD and SUB use a 9-bit adder so that the carry (or borrow) appears in bit 8. Branches and jumps are resolved in

this stage: the branch is taken if

$$take_branch = valid \wedge (jump \vee (br_z \wedge Z) \vee (br_nz \wedge \overline{Z})).$$

4.6 Memory access (MEM) and write back (WB)

The data memory is read asynchronously and written on the rising clock edge; Vivado maps it to distributed LUT-RAM. In MEM the result multiplexer chooses between the memory data (LD) and the ALU result. In WB the selected value is written into the register file.

4.7 Pipeline registers

Table 5 lists the contents of the four pipeline registers. Every register carries a `valid` bit, and every write-enable signal is AND-ed with it, so a bubble can never change the architectural state.

Table 5: Pipeline register contents

Register	Bits	Contents
IF/ID	17	valid, 16-bit instruction
ID/EX	48	valid, alu_op, alu_src_imm, uses_a/b, reg_we, mem_to_reg, mem_we, flag_we, jump, br_z, br_nz, halt, rd, ra, rb, A value, B value, imm8
EX/MEM	32	valid, reg_we, mem_to_reg, mem_we, halt, rd, ALU result, memory address, store data
MEM/WB	14	valid, reg_we, halt, rd, result

All pipeline registers and the PC are updated in a single clocked block with the following priority, which makes the behaviour in corner cases (for example a stall and a flush in the same cycle) well defined:

halted (freeze) > flush > HLT in pipeline > stall > normal advance.

5 Hazard Handling

5.1 Data hazards: forwarding

Consider the sequence used at the start of the test program:

```
LDI R1, 0x3C      ; R1 = 0x3C
LDI R2, 0x0F      ; R2 = 0x0F
AND R3, R1, R2    ; needs R2 (distance 1) and R1 (distance 2)
OR  R4, R1, R2    ; needs R2 (distance 2) and R1 (distance 3)
```

When `AND` reaches EX, the value of `R2` is still in the EX/MEM register and the value of `R1` is in the MEM/WB register; neither has been written into the register file yet. The hazard unit compares the source register numbers of the instruction in EX with the destination registers of the instructions in MEM and WB and sets `fwd_a_sel` and `fwd_b_sel`:

- `1` – take the EX/MEM result (distance 1). This path is not used if the instruction in MEM is a load, because the loaded byte is not available yet.
- `2` – take the MEM/WB result (distance 2).
- `0` – use the value read in ID (no hazard, or distance 3 which is handled by the register-file bypass).

EX/MEM has priority over MEM/WB because it holds the newer value. Only operands that are actually used (`uses_a`/`uses_b`) are compared, so unused register fields never cause a false forward. In the simulation trace (Figure 8) cycle 5 shows `fwdA=2 fwdB=1` for the `AND` instruction exactly as predicted, and cycle 6 shows `fwdA=0 fwdB=2` for `OR`, where `R1` is now obtained through the register-file bypass.

5.2 Load-use hazard: one-cycle stall

A load produces its data only at the end of the MEM stage. If the next instruction needs that data in EX, forwarding alone cannot help, because the data would have to travel backwards in time. The hazard unit therefore asserts `stall` when

$$stall = load_in_EX \wedge valid_ID \wedge ((uses_a \wedge ra_{ID} = rd_{EX}) \vee (uses_b \wedge rb_{ID} = rd_{EX})).$$

During a stall the PC and IF/ID keep their values and a bubble is inserted into ID/EX. In the next cycle the load is in WB and its value is forwarded from MEM/WB. Figure 5 shows the `LD R5,[R4]` / `ADDI R5,0xF0` pair of the test program (cycles 44–48 of the trace).

Cycle	44	45	46	47	48	49
`LD R5,[R4]`	ID	EX	MEM	WB		
`ADDI R5,0xF0`	IF	ID	ID	EX	MEM	WB
bubble			–	–	–	–
`ADDI R5,1`		IF	IF	ID	EX	MEM

Figure 5: Load-use hazard: `ADDI` waits one cycle in ID, then receives R5 from MEM/WB

5.3 Control hazards: flush

Branches and jumps are resolved in EX. By that time the two following instructions have already been fetched (one in IF, one in ID). If the branch is taken, these two instructions are wrong and must be cancelled: the PC is loaded with the target address, the IF/ID valid bit is cleared and a bubble is inserted into ID/EX. A taken branch therefore costs two cycles. A branch that is not taken costs nothing, because the pipeline simply continues with the sequential instructions (static *predict-not-taken*).

Cycle	8	9	10	11	12	13
`07: BZ 0x0A`	IF	ID	EX	MEM	WB	
`08: LDI R7,0xFF`		IF	–	–	–	–
`09: LDI R6,0x00`			–	–	–	–
`0A: LDI R1,5`				IF	ID	EX

Figure 6: Taken branch: the two wrong-path instructions are flushed (cycle 10, `flush=1`)

Because the flags are written at the end of the EX stage and the branch reads them in the next instruction's EX stage, the Z flag tested by a branch is always the flag produced by the immediately preceding flag-setting instruction, without any additional hazard logic.

5.4 Halt

When `HLT` reaches EX, fetching stops and younger instructions are squashed. When it reaches WB the `halted` output goes high and the whole pipeline freezes, so the final register and memory contents remain stable for inspection by the testbench.

6 Verification

6.1 Testbench

The testbench `tb_risc8_5stage.v` is self-checking:

- it generates a 100 MHz clock (10 ns period) and holds reset for two cycles;
- in every cycle it prints the PC, the IF/ID instruction, the valid bits of all pipeline registers, the `stall`, `flush` and forwarding signals, and the flags;
- it waits for `halted` (with a 2000-cycle timeout), then compares seven registers, one memory location and the flags with their expected values and prints `PASS` or `FAIL` for each;
- it prints statistics (cycles, instructions, CPI, stalls, taken branches and forwarded operands) and finally `*** ALL TESTS PASSED ***` or `*** TEST FAILED ***`.

6.2 Test program

The test program (Table 6) is stored in `imem.v`. It was written so that every hazard case occurs at least once: forwarding at distances 1, 2 and 3, a taken `BZ` with two wrong-path instructions, a loop closed by `BNZ` (taken four times, not taken once), a `JMP`, a store followed by a load, a load-use stall and a carry-producing addition.

Table 6: Test program

Addr	Code	Instruction	Purpose / result
00	823C	LDI R1, 0x3C	R1 = 0x3C
01	840F	LDI R2, 0x0F	R2 = 0x0F
02	3650	AND R3, R1, R2	R3 = 0x0C; forward R2 (d=1), R1 (d=2)
03	4850	OR R4, R1, R2	R4 = 0x3F; R1 via register-file bypass (d=3)
04	5CE0	XOR R6, R3, R4	R6 = 0x33
05	6D80	SHL R6, R6	R6 = 0x66, C = 0
06	2FB0	SUB R7, R6, R6	R7 = 0x00, Z = 1
07	D00A	BZ 0x0A	taken → flush 2 instructions
08	8EFF	LDI R7, 0xFF	wrong path (must not execute)
09	8C00	LDI R6, 0x00	wrong path (must not execute)
0A	8205	LDI R1, 5	loop counter
0B	8400	LDI R2, 0	sum
0C	1488	ADD R2, R2, R1	loop: R2 = 5+4+3+2+1
0D	92FF	ADDI R1, 0xFF	R1 = R1 − 1
0E	E00C	BNZ 0x0C	taken 4 times, not taken once
0F	8603	LDI R3, 0x03	
10	76C0	SHR R3, R3	R3 = 0x01, C = 1
11	C014	JMP 0x14	taken → flush 2 instructions
12	8E11	LDI R7, 0x11	wrong path
13	8699	LDI R3, 0x99	wrong path
14	8820	LDI R4, 0x20	address
15	B500	ST R2, [R4]	MEM[0x20] = 0x0F
16	AB00	LD R5, [R4]	R5 = 0x0F
17	9AF0	ADDI R5, 0xF0	load-use → 1 stall; R5 = 0xFF
18	9A01	ADDI R5, 1	R5 = 0x00, Z = 1, C = 1
19	F000	HLT	stop

6.3 Simulation results

The behavioural simulation was run in the Vivado simulator (XSim). Figure 7 shows the end of the Tcl Console: all nine checks pass and the simulation stops at 526 ns with the message `*** ALL TESTS PASSED ***`.

```
cyc  45 | PC=18 | IF/ID v 9af0 | ID/EX 1 EX/MEM 1 MEM/WB 1 | stall=1 flush=0 fwdA=2 fwdB=0 | Z=0 C=1
cyc  46 | PC=18 | IF/ID v 9af0 | ID/EX 0 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=1
cyc  47 | PC=19 | IF/ID v 9a01 | ID/EX 1 EX/MEM 0 MEM/WB 1 | stall=0 flush=0 fwdA=2 fwdB=0 | Z=0 C=1
cyc  48 | PC=1a | IF/ID v f000 | ID/EX 1 EX/MEM 1 MEM/WB 0 | stall=0 flush=0 fwdA=1 fwdB=0 | Z=0 C=0
cyc  49 | PC=1b | IF/ID v 0000 | ID/EX 1 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=1 C=1
cyc  50 | PC=1b | IF/ID - 0000 | ID/EX 0 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=1 C=1

==== Results ====
  PASS  R1 = 0x00
  PASS  R2 = 0x0f
  PASS  R3 = 0x01
  PASS  R4 = 0x20
  PASS  R5 = 0x00
  PASS  R6 = 0x66
  PASS  R7 = 0x00
  PASS  MEM[0x20] = 0x0f
  PASS  Z,C = 0x03

==== Statistics ====
  cycles            : 51
  instructions      : 34
  CPI               : 1.50
  load-use stalls   : 1
  taken branches    : 6 (2-cycle flush each)
  forwarded operands: 15

*** ALL TESTS PASSED ***
$finish called at time : 526 ns : File "C:/Users/PC/risc8_5stage/risc8_5stage.srcs/sim_1/new/tb_risc8_5stage.v" Line 127
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_risc8_5stage_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:05 ; elapsed = 00:00:07 . Memory (MB): peak = 1803.969 ; gain = 10.203
```

Figure 7: Simulation results and statistics printed by the testbench in the Vivado Tcl Console

Table 7 compares the final state with the expected values. The two wrong-path instructions after `BZ` would have written `R7 = 0xFF` and `R6 = 0x00`, and those after `JMP` would have written `R7 = 0x11` and `R3 = 0x99`; the fact that `R3`, `R6` and `R7` hold the correct values proves that the flush works.

Table 7: Final architectural state

Item	Expected	Simulated	Result	What it proves
R1	0x00	0x00	PASS	loop counter reached 0 (BNZ loop)
R2	0x0F	0x0F	PASS	5+4+3+2+1 = 15, forwarding in loop
R3	0x01	0x01	PASS	SHR, wrong path after JMP flushed
R4	0x20	0x20	PASS	LDI
R5	0x00	0x00	PASS	LD, load-use stall, ADDI with carry
R6	0x66	0x66	PASS	XOR/SHL, wrong path after BZ flushed
R7	0x00	0x00	PASS	SUB, both wrong paths flushed
MEM[0x20]	0x0F	0x0F	PASS	ST
{Z, C}	11_2	11_2	PASS	flag generation

6.4 Cycle-by-cycle trace

Figures 8 and 9 show the per-cycle trace. The following events can be identified:

- **Cycles 5–9**: non-zero `fwdA`/`fwdB` values – forwarding for the dependent AND, OR, XOR, SHL and SUB instructions.
- **Cycle 10**: `flush=1` – `BZ 0x0A` is taken; in cycle 11 the IF/ID register is marked invalid (`-`) and the PC continues from `0x0A`.
- **Cycles 17, 22, 27, 32**: `flush=1` – the four taken `BNZ` branches of the loop (PC returns from `0x10` to `0x0C`).
- **Cycle 37**: the loop exits; `Z=1` and no flush.
- **Cycle 40**: `flush=1` – `JMP 0x14`.

- **Cycle 45**: `stall=1` – the load-use hazard; PC and IF/ID hold `0x18/9AF0` for one more cycle, and in cycle 46 ID/EX contains a bubble.
- **Cycles 49–50**: HLT reaches the end of the pipeline and the processor halts.

```
Time resolution is 1 ps
source tb_risc8_5stage.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [llength [get_objects]] > 0} {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#      send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave w
#   }
# }
# run 1000ns
==== risc8 5-stage pipeline simulation ====
cyc   1 | PC=00 | IF/ID - 0000 | ID/EX 0 EX/MEM 0 MEM/WB 0 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=0
cyc   2 | PC=01 | IF/ID v 823c | ID/EX 0 EX/MEM 0 MEM/WB 0 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=0
cyc   3 | PC=02 | IF/ID v 840f | ID/EX 1 EX/MEM 0 MEM/WB 0 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=0
cyc   4 | PC=03 | IF/ID v 3650 | ID/EX 1 EX/MEM 1 MEM/WB 0 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=0
cyc   5 | PC=04 | IF/ID v 4850 | ID/EX 1 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=2 fwdB=1 | Z=0 C=0
cyc   6 | PC=05 | IF/ID v 5ce0 | ID/EX 1 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=2 | Z=0 C=0
cyc   7 | PC=06 | IF/ID v 6d80 | ID/EX 1 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=2 fwdB=1 | Z=0 C=0
cyc   8 | PC=07 | IF/ID v 2fb0 | ID/EX 1 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=1 fwdB=0 | Z=0 C=0
cyc   9 | PC=08 | IF/ID v d00a | ID/EX 1 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=1 fwdB=1 | Z=0 C=0
cyc  10 | PC=09 | IF/ID v 8eff | ID/EX 1 EX/MEM 1 MEM/WB 1 | stall=0 flush=1 fwdA=0 fwdB=0 | Z=1 C=0
cyc  11 | PC=0a | IF/ID - 8eff | ID/EX 0 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=1 C=0
cyc  12 | PC=0b | IF/ID v 8205 | ID/EX 0 EX/MEM 0 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=1 C=0
cyc  13 | PC=0c | IF/ID v 8400 | ID/EX 1 EX/MEM 0 MEM/WB 0 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=1 C=0
cyc  14 | PC=0d | IF/ID v 1488 | ID/EX 1 EX/MEM 1 MEM/WB 0 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=1 C=0
cyc  15 | PC=0e | IF/ID v 92ff | ID/EX 1 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=1 fwdB=2 | Z=1 C=0
cyc  16 | PC=0f | IF/ID v e00c | ID/EX 1 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=0
cyc  17 | PC=10 | IF/ID v 8603 | ID/EX 1 EX/MEM 1 MEM/WB 1 | stall=0 flush=1 fwdA=0 fwdB=0 | Z=0 C=1
cyc  18 | PC=0c | IF/ID - 8603 | ID/EX 0 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=1
cyc  19 | PC=0d | IF/ID v 1488 | ID/EX 0 EX/MEM 0 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=1
```

Figure 8: Simulation trace, cycles 1–19 (forwarding, BZ flush and the first loop iterations)

```
cyc  19 | PC=0d | IF/ID v 1488 | ID/EX 0 EX/MEM 0 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=1
cyc  20 | PC=0e | IF/ID v 92ff | ID/EX 1 EX/MEM 0 MEM/WB 0 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=1
cyc  21 | PC=0f | IF/ID v e00c | ID/EX 1 EX/MEM 1 MEM/WB 0 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=0
cyc  22 | PC=10 | IF/ID v 8603 | ID/EX 1 EX/MEM 1 MEM/WB 1 | stall=0 flush=1 fwdA=0 fwdB=0 | Z=0 C=1
cyc  23 | PC=0c | IF/ID - 8603 | ID/EX 0 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=1
cyc  24 | PC=0d | IF/ID v 1488 | ID/EX 0 EX/MEM 0 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=1
cyc  25 | PC=0e | IF/ID v 92ff | ID/EX 1 EX/MEM 0 MEM/WB 0 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=1
cyc  26 | PC=0f | IF/ID v e00c | ID/EX 1 EX/MEM 1 MEM/WB 0 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=0
cyc  27 | PC=10 | IF/ID v 8603 | ID/EX 1 EX/MEM 1 MEM/WB 1 | stall=0 flush=1 fwdA=0 fwdB=0 | Z=0 C=1
cyc  28 | PC=0c | IF/ID - 8603 | ID/EX 0 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=1
cyc  29 | PC=0d | IF/ID v 1488 | ID/EX 0 EX/MEM 0 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=1
cyc  30 | PC=0e | IF/ID v 92ff | ID/EX 1 EX/MEM 0 MEM/WB 0 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=1
cyc  31 | PC=0f | IF/ID v e00c | ID/EX 1 EX/MEM 1 MEM/WB 0 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=0
cyc  32 | PC=10 | IF/ID v 8603 | ID/EX 1 EX/MEM 1 MEM/WB 1 | stall=0 flush=1 fwdA=0 fwdB=0 | Z=0 C=1
cyc  33 | PC=0c | IF/ID - 8603 | ID/EX 0 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=1
cyc  34 | PC=0d | IF/ID v 1488 | ID/EX 0 EX/MEM 0 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=1
cyc  35 | PC=0e | IF/ID v 92ff | ID/EX 1 EX/MEM 0 MEM/WB 0 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=1
cyc  36 | PC=0f | IF/ID v e00c | ID/EX 1 EX/MEM 1 MEM/WB 0 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=0
cyc  37 | PC=10 | IF/ID v 8603 | ID/EX 1 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=1 C=1
cyc  38 | PC=11 | IF/ID v 76c0 | ID/EX 1 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=1 C=1
cyc  39 | PC=12 | IF/ID v c014 | ID/EX 1 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=1 fwdB=0 | Z=1 C=1
cyc  40 | PC=13 | IF/ID v 8e11 | ID/EX 1 EX/MEM 1 MEM/WB 1 | stall=0 flush=1 fwdA=0 fwdB=0 | Z=0 C=1
cyc  41 | PC=14 | IF/ID - 8e11 | ID/EX 0 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=1
cyc  42 | PC=15 | IF/ID v 8820 | ID/EX 0 EX/MEM 0 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=1
cyc  43 | PC=16 | IF/ID v b500 | ID/EX 1 EX/MEM 0 MEM/WB 0 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=1
cyc  44 | PC=17 | IF/ID v ab00 | ID/EX 1 EX/MEM 1 MEM/WB 0 | stall=0 flush=0 fwdA=1 fwdB=0 | Z=0 C=1
cyc  45 | PC=18 | IF/ID v 9af0 | ID/EX 1 EX/MEM 1 MEM/WB 1 | stall=1 flush=0 fwdA=2 fwdB=0 | Z=0 C=1
cyc  46 | PC=18 | IF/ID v 9af0 | ID/EX 0 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=0 C=1
cyc  47 | PC=19 | IF/ID v 9a01 | ID/EX 1 EX/MEM 0 MEM/WB 1 | stall=0 flush=0 fwdA=2 fwdB=0 | Z=0 C=1
cyc  48 | PC=1a | IF/ID v f000 | ID/EX 1 EX/MEM 1 MEM/WB 0 | stall=0 flush=0 fwdA=1 fwdB=0 | Z=0 C=0
cyc  49 | PC=1b | IF/ID v 0000 | ID/EX 1 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=1 C=1
cyc  50 | PC=1b | IF/ID - 0000 | ID/EX 0 EX/MEM 1 MEM/WB 1 | stall=0 flush=0 fwdA=0 fwdB=0 | Z=1 C=1
```

Figure 9: Simulation trace, cycles 19–50 (loop, JMP flush and load-use stall)

6.5 Waveform

Figure 10 shows the waveform of the main top-level signals and of the testbench counters. `rst` is high for the first two cycles; `pc_out` advances and jumps backwards in the loop; `n_flush` increments from 0 to 6 at each taken branch; `n_stall` becomes 1 at about 450 ns (the load-use stall); `errors` stays 0; and `halted` rises at the end of the program.

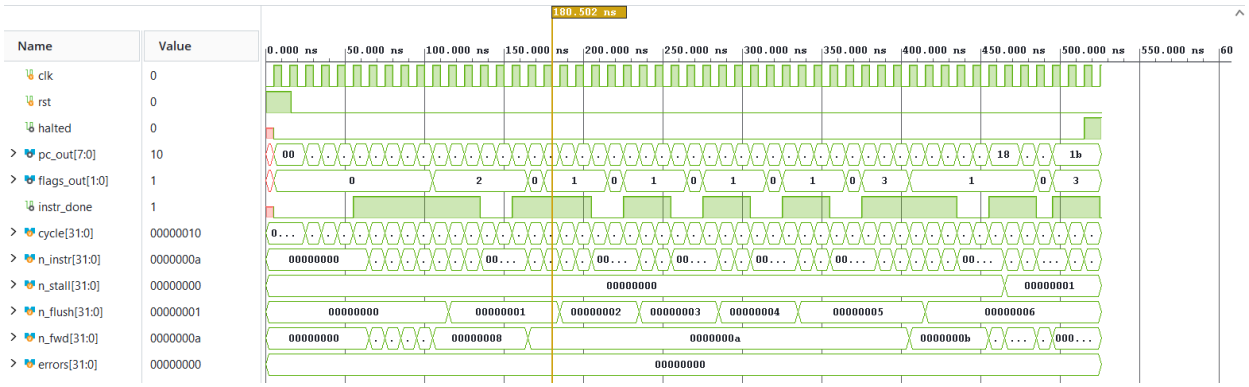


Figure 10: Behavioural simulation waveform in Vivado (0–526 ns)

6.6 Performance analysis

The program executes 34 instructions in 51 cycles. The extra cycles can be explained exactly:

$$\underbrace{34}_{\text{instructions}} + \underbrace{4}_{\text{pipeline fill}} + \underbrace{1}_{\text{load-use stall}} + \underbrace{6 \times 2}_{\text{taken branches}} = 51 \text{ cycles}, \qquad \text{CPI} = \frac{51}{34} = 1.50.$$

The pipeline-fill cost is paid only once, so for long programs the CPI approaches $1 + (\text{stalls} + 2 \times \text{taken branches})/\text{instructions}$. Without forwarding, each of the 15 forwarded operands would have required up to two stall cycles, so forwarding removes a large fraction of the potential stall cycles. A non-pipelined multi-cycle design using five cycles per instruction would need $34 \times 5 = 170$ cycles for the same program, i.e. the pipeline is about $3.3\times$ faster at the same clock frequency.

6.7 Additional verification

In addition to the directed test, the design was cross-checked during development with an open-source simulator (Icarus Verilog) against an instruction-level reference model written in Python: 500 randomly generated programs (random instructions, registers, immediates and forward branches) produced register, memory and flag states identical to the reference model. The RTL was also checked with the Verilator linter with all warnings enabled, which reported no warnings.

7 Synthesis and Implementation Results

7.1 Project settings

The project targets the Artix-7 device `xc7a35tcpg236-1` (the device of the Digilent Basys-3 board). The only constraint is a 100 MHz clock on the `clk` port:

```
create_clock -period 10.000 -name sys_clk -waveform {0.000 5.000} [get_ports clk]
```

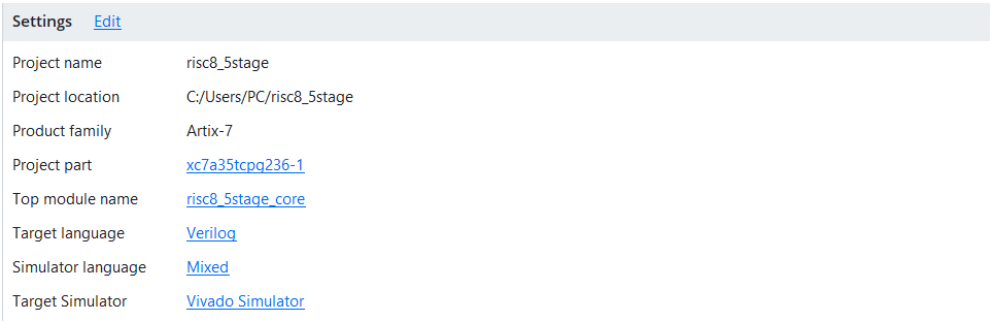


Figure 11: Vivado project settings

7.2 Resource utilisation

Synthesis and implementation completed without errors (Figure 12). Table 8 summarises the resources used after placement.

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes
✓ synth_1	constrs_1	synth_design Complete!								
✓ impl_1	constrs_1	route_design Complete!	1.747	0.000	0.040	0.000		0.000	0.084	0

(a) Status, timing and power columns

Name	Constraints	Methodology	RQA Score	QoR Suggestions	LUT	FF	BRAM	URAM	DSP
✓ synth_1	constrs_1				242	186	0	0	0
✓ impl_1	constrs_1	13 Warn			240	186	0	0	0

(b) Methodology check and resource columns

Figure 12: Vivado Design Runs: synthesis (`synth_1`) and implementation (`impl_1`) completed

Table 8: Resource utilisation on `xc7a35tcpg236-1`

Resource	After synthesis	After implementation	Available	Utilisation
Slice LUTs	242	240	20 800	1.2 %
Flip-flops (FF)	186	186	41 600	0.4 %
LUT as memory	data memory (8 × RAM256X1S)		9 600	≈1 %
Block RAM	0	0	50	0 %
DSP slices	0	0	90	0 %
I/O pins	14	14	106	13 %
BUFG	1	1	32	3 %

The core is very small: about 1 % of the logic of this low-cost device. The 186 flip-flops correspond to the PC (8), the flags and halt state (3), the four pipeline registers ($17 + 48 + 32 + 14 = 111$) and the eight 8-bit registers of the register file (64). The data memory is implemented in distributed LUT-RAM rather than block RAM, because it is read asynchronously. No DSP blocks are needed because the ALU only adds and subtracts. The I/O count (14) corresponds to `clk`, `rst`, `pc_out[7:0]`, `flags_out[1:0]`, `halted` and `instr_done`.

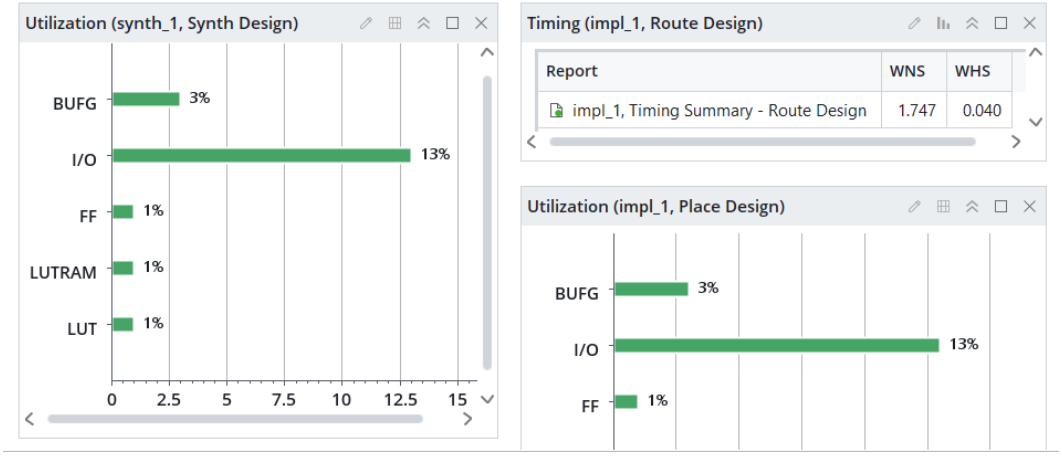


Figure 13: Vivado dashboard: utilisation after synthesis and placement, and timing summary

7.3 Timing

Table 9 shows the timing summary after routing.

Table 9: Timing summary after routing (100 MHz constraint)

Metric	Value	Meaning
WNS (worst negative slack)	+1.747 ns	setup requirement met with 1.747 ns margin
TNS (total negative slack)	0.000 ns	no failing setup paths
WHS (worst hold slack)	+0.040 ns	hold requirement met
THS (total hold slack)	0.000 ns	no failing hold paths
TPWS (total pulse width slack)	0.000 ns	no pulse-width violations
Failed routes	0	fully routed

A positive WNS means that the longest register-to-register path is shorter than the 10 ns clock period. The critical path delay is therefore about $10 - 1.747 = 8.253$ ns, which gives an estimated maximum clock frequency of

$$f_{\max} \approx \frac{1}{10\,\text{ns} - 1.747\,\text{ns}} = \frac{1}{8.253\,\text{ns}} \approx 121\ \text{MHz}.$$

The critical path is expected to run from a pipeline register through the forwarding multiplexer and the ALU carry chain to the flag register or the branch logic, which is the longest combinational path in the EX stage.

7.4 Power

Vivado estimates a total on-chip power of 0.084 W with default switching activity. For such a small design on this device, this figure is dominated by the static (leakage) power of the FPGA rather than by the processor logic itself.

7.5 Methodology warnings

The implementation run reports 13 methodology warnings. Because the design is not mapped to a board, the ports have no package-pin assignments and no input/output delay constraints; warnings of this kind are expected in this situation and do not affect the internal register-to-register timing reported above. They would be resolved by adding pin and I/O constraints when the core is placed on a real board.

8 Discussion

8.1 Design decisions

- **Valid bits instead of NOP injection.** Every pipeline register has a valid bit and all write enables are qualified by it. Bubbles are created by clearing this bit, which keeps the stall and flush logic simple and safe.
- **uses_a / uses_b signals.** Instructions such as LDI or JMP do not read registers, but their encoding still contains bits in the register fields. Comparing only the operands that are really used avoids unnecessary stalls and forwards.
- **Write-through register file.** The distance-3 hazard is solved in the register file, so the EX forwarding multiplexers only need three inputs.
- **Single clocked block with explicit priority.** All sequential state is updated in one `always @(posedge clk)` block with a clear priority order, which makes corner cases deterministic and easy to review.

8.2 Limitations

- The address space is limited to 256 instructions and 256 data bytes, and there is no stack, subroutine call, interrupt or I/O instruction.
- Branches are resolved in EX and always cost two cycles when taken.
- The program is built into the ROM, so a new program requires re-synthesis.
- The design was verified in simulation only and not on a physical board.

8.3 Future work

- Resolve branches in ID or add a simple branch predictor to reduce the branch penalty.
- Add `CALL`/`RET` and a stack pointer, and memory-mapped I/O.

- Load programs from a `.hex` file with `$readmemh` or through a UART bootloader.
- Add a board wrapper with a clock divider to display the PC and flags on LEDs.

9 Conclusion

An 8-bit RISC processor with a five-stage pipeline was designed in Verilog, verified and implemented in AMD Vivado 2026.1. The processor supports 16 instructions and resolves all data and control hazards in hardware using forwarding, a write-through register file, a one-cycle load-use stall and a two-cycle branch flush. A self-checking testbench confirmed that the 34-instruction test program, which exercises every hazard case, produces the correct register, memory and flag values, with a CPI of 1.50 that is fully explained by pipeline fill, one stall and six taken branches. On an Artix-7 `xc7a35t` device the core uses only 240 LUTs and 186 flip-flops and meets the 100 MHz constraint with a positive slack of 1.747 ns (about 121 MHz maximum). The project demonstrates the main concepts of pipelined processor design and the complete FPGA design flow from RTL to timing closure.

References

[1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, RISC-V ed., Morgan Kaufmann.

[2] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture*, Morgan Kaufmann.

[3] AMD, *Vivado Design Suite User Guide: Synthesis (UG901)*.

[4] AMD, *Vivado Design Suite User Guide: Logic Simulation (UG900)*.

[5] AMD, *Vivado Design Suite User Guide: Using Constraints (UG903)*.

A Source Code

A.1 design_sources/alu.v

```
// ============================================================================
// Lê Kỳ Dũng - mã số sinh viên SE190745
// ----------------------------------------------------------------------------
// alu.v  -  8-bit Arithmetic Logic Unit  (Stage 3, EX)
// ----------------------------------------------------------------------------
// Purely combinational. Computes the result and the Z / C flags.
//
//   op  operation          C output
//   0   ADD  y = a + b     carry out of bit 7
//   1   SUB  y = a - b     borrow (1 when a < b)
//   2   AND  y = a & b     unchanged (c_in)
//   3   OR   y = a | b     unchanged (c_in)
//   4   XOR  y = a ^ b     unchanged (c_in)
//   5   SHL  y = a << 1    old bit 7
//   6   SHR  y = a >> 1    old bit 0
//   7   PASS y = b         unchanged (c_in)   (used by LDI)
//
// Z is 1 whenever the result is zero.
// ============================================================================
`timescale 1ns / 1ps

module alu (
    input  wire [7:0] a,        // operand A
    input  wire [7:0] b,        // operand B (register or immediate)
    input  wire [2:0] op,       // operation select
    input  wire       c_in,     // current carry flag (kept by logic ops)
    output reg  [7:0] y,        // result
    output wire       z,        // zero flag
    output reg        c         // carry / borrow flag
);

    // 9-bit sum and difference so that the carry / borrow appears in bit 8
    wire [8:0] sum  = {1'b0, a} + {1'b0, b};
    wire [8:0] diff = {1'b0, a} - {1'b0, b};

    always @(*) begin
        // defaults: avoid latches
        y = 8'h00;
        c = c_in;
        case (op)
            3'd0: begin y = sum[7:0];          c = sum[8];  end  // ADD
            3'd1: begin y = diff[7:0];         c = diff[8]; end  // SUB (borrow)
            3'd2: begin y = a & b;                          end  // AND
            3'd3: begin y = a | b;                          end  // OR
            3'd4: begin y = a ^ b;                          end  // XOR
            3'd5: begin y = {a[6:0], 1'b0};    c = a[7];    end  // SHL
            3'd6: begin y = {1'b0, a[7:1]};    c = a[0];    end  // SHR
            default: begin y = b;                           end  // PASS B
        endcase
    end

    assign z = (y == 8'h00);

endmodule
```

A.2 design_sources/regfile.v

```
// ============================================================================
// Lê Kỳ Dũng - mã số sinh viên SE190745
// ----------------------------------------------------------------------------
// regfile.v  -  8 x 8-bit register file  (read in ID, written in WB)
// ----------------------------------------------------------------------------
// * Two asynchronous read ports (A and B), one synchronous write port.
// * Write-through bypass: if WB writes the register that ID is reading in the
//   same cycle, the new value is returned immediately. This removes the
//   distance-3 data hazard without any stall.
```

```
// * R0 is an ordinary register (it is NOT hard-wired to zero).
// ============================================================================
`timescale 1ns / 1ps

module regfile (
    input  wire       clk,
    input  wire       rst,
    // write port (from WB)
    input  wire       we,
    input  wire [2:0] waddr,
    input  wire [7:0] wdata,
    // read port A
    input  wire [2:0] ra_addr,
    output wire [7:0] ra_data,
    // read port B
    input  wire [2:0] rb_addr,
    output wire [7:0] rb_data
);

    reg [7:0] regs [0:7];
    integer i;

    // synchronous write, synchronous reset
    always @(posedge clk) begin
        if (rst) begin
            for (i = 0; i < 8; i = i + 1)
                regs[i] <= 8'h00;
        end else if (we) begin
            regs[waddr] <= wdata;
        end
    end

    // asynchronous read with write-through bypass
    assign ra_data = (we && (waddr == ra_addr)) ? wdata : regs[ra_addr];
    assign rb_data = (we && (waddr == rb_addr)) ? wdata : regs[rb_addr];

endmodule
```

A.3 design_sources/imem.v

```
// ============================================================================
// Lê Kỳ Dũng - mã số sinh viên SE190745
// ----------------------------------------------------------------------------
// imem.v  -  256 x 16-bit instruction ROM  (Stage 1, IF)
// ----------------------------------------------------------------------------
// The test program is built into this file, so Vivado can never lose it.
// Asynchronous read: the instruction appears in the same cycle as the PC.
//
// Encoding reminder
//   R-type : opcode[15:12] rd[11:9] rs1[8:6] rs2[5:3] 000
//   R1-type: opcode[15:12] rd[11:9] rs1[8:6] 000000
//   I-type : opcode[15:12] rd[11:9] 0        imm8[7:0]
//   J-type : opcode[15:12] 0000              addr8[7:0]
//
// Opcodes: 0 NOP 1 ADD 2 SUB 3 AND 4 OR 5 XOR 6 SHL 7 SHR
//          8 LDI 9 ADDI A LD B ST C JMP D BZ E BNZ F HLT
//
// To run your own program: replace the case items below (unused addresses
// return NOP). Remember to update the expected values in the testbench.
// ============================================================================
`timescale 1ns / 1ps

module imem (
    input  wire [7:0]  addr,
    output reg  [15:0] instr
);

    always @(*) begin
        case (addr)
            // ---- forwarding at distances 1, 2 and 3 -------------------------
            8'h00: instr = 16'h823C; // LDI  R1, 0x3C      R1 = 0x3C
            8'h01: instr = 16'h840F; // LDI  R2, 0x0F      R2 = 0x0F
```

```
            8'h02: instr = 16'h3650; // AND  R3, R1, R2    R3 = 0x0C (R2 d1, R1 d2)
            8'h03: instr = 16'h4850; // OR   R4, R1, R2    R4 = 0x3F (R1 d3 bypass)
            8'h04: instr = 16'h5CE0; // XOR  R6, R3, R4    R6 = 0x33
            8'h05: instr = 16'h6D80; // SHL  R6, R6        R6 = 0x66, C = 0
            8'h06: instr = 16'h2FB0; // SUB  R7, R6, R6    R7 = 0x00, Z = 1
            // ---- taken BZ with two wrong-path instructions ------------------
            8'h07: instr = 16'hD00A; // BZ   0x0A          taken
            8'h08: instr = 16'h8EFF; // LDI  R7, 0xFF      wrong path (flushed)
            8'h09: instr = 16'h8C00; // LDI  R6, 0x00      wrong path (flushed)
            // ---- loop: R2 = 5 + 4 + 3 + 2 + 1 -------------------------------
            8'h0A: instr = 16'h8205; // LDI  R1, 5
            8'h0B: instr = 16'h8400; // LDI  R2, 0
            8'h0C: instr = 16'h1488; // loop: ADD R2, R2, R1
            8'h0D: instr = 16'h92FF; //       ADDI R1, 0xFF   (R1 - 1)
            8'h0E: instr = 16'hE00C; //       BNZ  loop       taken 4 times
            // ---- shift right with carry, then a jump ----------------------
            8'h0F: instr = 16'h8603; // LDI  R3, 0x03
            8'h10: instr = 16'h76C0; // SHR  R3, R3        R3 = 0x01, C = 1
            8'h11: instr = 16'hC014; // JMP  0x14          taken
            8'h12: instr = 16'h8E11; // LDI  R7, 0x11      wrong path (flushed)
            8'h13: instr = 16'h8699; // LDI  R3, 0x99      wrong path (flushed)
            // ---- store, load, load-use stall, carry -------------------------
            8'h14: instr = 16'h8820; // LDI  R4, 0x20
            8'h15: instr = 16'hB500; // ST   R2, [R4]      mem[0x20] = 0x0F
            8'h16: instr = 16'hAB00; // LD   R5, [R4]      R5 = 0x0F
            8'h17: instr = 16'h9AF0; // ADDI R5, 0xF0      load-use: 1 stall, R5 = 0xFF
            8'h18: instr = 16'h9A01; // ADDI R5, 1         R5 = 0x00, Z = 1, C = 1
            8'h19: instr = 16'hF000; // HLT
            default: instr = 16'h0000; // NOP
        endcase
    end

endmodule
```

A.4 design_sources/dmem.v

```
// ============================================================================
// Lê Kỳ Dũng - mã số sinh viên SE190745
// ----------------------------------------------------------------------------
// dmem.v  -  256 x 8-bit data memory  (Stage 4, MEM)
// ----------------------------------------------------------------------------
// * Asynchronous read  (LD gets its byte in the same cycle as the address).
// * Synchronous write  (ST writes on the rising clock edge).
// Vivado maps this to distributed LUT-RAM.
// ============================================================================
`timescale 1ns / 1ps

module dmem (
    input  wire       clk,
    input  wire       we,       // write enable (ST in MEM)
    input  wire [7:0] addr,     // byte address
    input  wire [7:0] wdata,    // store data
    output wire [7:0] rdata     // load data
);

    reg [7:0] mem [0:255];
    integer i;

    // all locations start at zero (valid for simulation and FPGA init)
    initial begin
        for (i = 0; i < 256; i = i + 1)
            mem[i] = 8'h00;
    end

    always @(posedge clk) begin
        if (we)
            mem[addr] <= wdata;
    end

    assign rdata = mem[addr];

endmodule
```

A.5 design_sources/control_unit.v

```
// ============================================================================
// Lê Kỳ Dũng - mã số sinh viên SE190745
// ----------------------------------------------------------------------------
// control_unit.v  -  hard-wired decoder  (Stage 2, ID)
// ----------------------------------------------------------------------------
// Purely combinational lookup table: opcode -> control signals.
// Every signal starts at 0 (= NOP) and each instruction switches on only what
// it needs, so no latches can be inferred.
//
// alu_op: 0 ADD 1 SUB 2 AND 3 OR 4 XOR 5 SHL 6 SHR 7 PASS-B
// ============================================================================
`timescale 1ns / 1ps

module control_unit (
    input  wire [3:0] opcode,
    output reg  [2:0] alu_op,       // ALU operation
    output reg        alu_src_imm,  // ALU B input = imm8
    output reg        ra_is_rd,     // read port A reads rd (ADDI)
    output reg        rb_is_rd,     // read port B reads rd (ST)
    output reg        uses_a,       // instruction really reads port A
    output reg        uses_b,       // instruction really reads port B
    output reg        reg_we,       // write rd in WB
    output reg        mem_to_reg,   // result comes from data memory (LD)
    output reg        mem_we,       // write data memory in MEM (ST)
    output reg        flag_we,      // update Z and C in EX
    output reg        jump,         // JMP
    output reg        br_z,         // BZ
    output reg        br_nz,        // BNZ
    output reg        halt          // HLT
);

    // opcode constants
    localparam OP_NOP  = 4'h0, OP_ADD = 4'h1, OP_SUB = 4'h2, OP_AND  = 4'h3,
               OP_OR   = 4'h4, OP_XOR = 4'h5, OP_SHL = 4'h6, OP_SHR  = 4'h7,
               OP_LDI  = 4'h8, OP_ADDI= 4'h9, OP_LD  = 4'hA, OP_ST   = 4'hB,
               OP_JMP  = 4'hC, OP_BZ  = 4'hD, OP_BNZ = 4'hE, OP_HLT  = 4'hF;

    always @(*) begin
        // ---- defaults: everything off (NOP) ----
        alu_op      = 3'd0;
        alu_src_imm = 1'b0;
        ra_is_rd    = 1'b0;
        rb_is_rd    = 1'b0;
        uses_a      = 1'b0;
        uses_b      = 1'b0;
        reg_we      = 1'b0;
        mem_to_reg  = 1'b0;
        mem_we      = 1'b0;
        flag_we     = 1'b0;
        jump        = 1'b0;
        br_z        = 1'b0;
        br_nz       = 1'b0;
        halt        = 1'b0;

        case (opcode)
            OP_ADD: begin alu_op = 3'd0; uses_a = 1'b1; uses_b = 1'b1;
                          reg_we = 1'b1; flag_we = 1'b1; end
            OP_SUB: begin alu_op = 3'd1; uses_a = 1'b1; uses_b = 1'b1;
                          reg_we = 1'b1; flag_we = 1'b1; end
            OP_AND: begin alu_op = 3'd2; uses_a = 1'b1; uses_b = 1'b1;
                          reg_we = 1'b1; flag_we = 1'b1; end
            OP_OR : begin alu_op = 3'd3; uses_a = 1'b1; uses_b = 1'b1;
                          reg_we = 1'b1; flag_we = 1'b1; end
            OP_XOR: begin alu_op = 3'd4; uses_a = 1'b1; uses_b = 1'b1;
                          reg_we = 1'b1; flag_we = 1'b1; end
            OP_SHL: begin alu_op = 3'd5; uses_a = 1'b1;
                          reg_we = 1'b1; flag_we = 1'b1; end
            OP_SHR: begin alu_op = 3'd6; uses_a = 1'b1;
                          reg_we = 1'b1; flag_we = 1'b1; end
            OP_LDI: begin alu_op = 3'd7; alu_src_imm = 1'b1;
                          reg_we = 1'b1; end
            OP_ADDI:begin alu_op = 3'd0; alu_src_imm = 1'b1; ra_is_rd = 1'b1;
```

```
                        uses_a = 1'b1; reg_we = 1'b1; flag_we = 1'b1; end
            OP_LD : begin uses_a = 1'b1; reg_we = 1'b1; mem_to_reg = 1'b1; end
            OP_ST : begin rb_is_rd = 1'b1; uses_a = 1'b1; uses_b = 1'b1;
                        mem_we = 1'b1; end
            OP_JMP: begin jump  = 1'b1; end
            OP_BZ : begin br_z  = 1'b1; end
            OP_BNZ: begin br_nz = 1'b1; end
            OP_HLT: begin halt  = 1'b1; end
            OP_NOP: ;   // all signals stay 0
            default: ;
        endcase
    end

endmodule
```

A.6 design_sources/hazard_unit.v

```
// ============================================================================
// Lê Kỳ Dũng - mã số sinh viên SE190745
// ----------------------------------------------------------------------------
// hazard_unit.v  -  forwarding selection and load-use stall
// ----------------------------------------------------------------------------
// Purely combinational.
//
// Forwarding (for the instruction in EX, separately for operand A and B):
//   1. MEM stage writes this register and is NOT a load -> take EX/MEM result
//   2. else WB stage writes this register               -> take MEM/WB result
//   3. else                                             -> value read in ID
//   fwd_*_sel : 2'd0 = ID value, 2'd1 = EX/MEM, 2'd2 = MEM/WB
//
// Load-use stall:
//   a load is in EX and the valid instruction in ID reads the register the
//   load will write -> stall = 1 for one cycle (hold PC and IF/ID, bubble
//   into ID/EX).
//
// Only operands the instruction really uses (uses_a / uses_b) are compared,
// so unused register fields never cause a false forward or stall.
// ============================================================================
`timescale 1ns / 1ps

module hazard_unit (
    // instruction in ID
    input  wire       ifid_valid,
    input  wire [2:0] id_ra,
    input  wire [2:0] id_rb,
    input  wire       id_uses_a,
    input  wire       id_uses_b,
    // instruction in EX
    input  wire       idex_valid,
    input  wire [2:0] idex_ra,
    input  wire [2:0] idex_rb,
    input  wire       idex_uses_a,
    input  wire       idex_uses_b,
    input  wire [2:0] idex_rd,
    input  wire       idex_reg_we,
    input  wire       idex_mem_to_reg,
    // instruction in MEM
    input  wire       exmem_valid,
    input  wire [2:0] exmem_rd,
    input  wire       exmem_reg_we,
    input  wire       exmem_mem_to_reg,
    // instruction in WB
    input  wire       memwb_valid,
    input  wire [2:0] memwb_rd,
    input  wire       memwb_reg_we,
    // outputs
    output reg  [1:0] fwd_a_sel,
    output reg  [1:0] fwd_b_sel,
    output wire       stall
);

    // can the MEM / WB stage supply a value?
```

```
    wire mem_can_fwd = exmem_valid & exmem_reg_we & ~exmem_mem_to_reg;
    wire wb_can_fwd  = memwb_valid & memwb_reg_we;

    // ---------------- operand A ----------------
    always @(*) begin
        fwd_a_sel = 2'd0;
        if (idex_valid && idex_uses_a) begin
            if (mem_can_fwd && (exmem_rd == idex_ra))
                fwd_a_sel = 2'd1;            // newest value wins
            else if (wb_can_fwd && (memwb_rd == idex_ra))
                fwd_a_sel = 2'd2;
        end
    end

    // ---------------- operand B ----------------
    always @(*) begin
        fwd_b_sel = 2'd0;
        if (idex_valid && idex_uses_b) begin
            if (mem_can_fwd && (exmem_rd == idex_rb))
                fwd_b_sel = 2'd1;
            else if (wb_can_fwd && (memwb_rd == idex_rb))
                fwd_b_sel = 2'd2;
        end
    end

    // ---------------- load-use stall ----------------
    wire load_in_ex = idex_valid & idex_reg_we & idex_mem_to_reg;

    assign stall = load_in_ex & ifid_valid &
                   ((id_uses_a & (id_ra == idex_rd)) |
                    (id_uses_b & (id_rb == idex_rd)));

endmodule
```

A.7 design_sources/risc8_5stage_core.v

```
// ============================================================================
// Lê Kỳ Dũng - mã số sinh viên SE190745
// ----------------------------------------------------------------------------
// risc8_5stage_core.v  -  top level of the 8-bit 5-stage RISC processor
// ----------------------------------------------------------------------------
//   IF  -> IF/ID -> ID -> ID/EX -> EX -> EX/MEM -> MEM -> MEM/WB -> WB
//
// Contains: PC, the four pipeline registers, the Z/C flag register, the
// branch and halt logic, the forwarding multiplexers, and instantiates
//   imem, control_unit, regfile, hazard_unit, alu, dmem.
//
// Pipeline-register update priority (one clocked block):
//   halted (freeze)  >  flush (taken branch)  >  halt in pipe  >  stall  >  normal
//
// Hazards
//   * RAW at distance 1/2 : forwarding EX/MEM -> EX and MEM/WB -> EX
//   * RAW at distance 3   : register-file write-through bypass
//   * load-use            : 1-cycle stall, then MEM/WB forwarding
//   * taken branch / jump : resolved in EX, 2 younger instructions flushed
// ============================================================================
`timescale 1ns / 1ps

module risc8_5stage_core (
    input  wire       clk,
    input  wire       rst,          // synchronous, active high
    output wire       halted,       // HLT has reached WB; pipeline frozen
    output wire [7:0] pc_out,       // current PC
    output wire [1:0] flags_out,    // {Z, C}
    output wire       instr_done    // 1 when a valid instruction is in WB
);

    // ========================================================================
    // Pipeline registers and architectural state
    // ========================================================================
    reg  [7:0]  pc;
    reg         z_flag, c_flag;
```

```
    reg         halt_done;          // sticky: HLT already counted as done

    // IF/ID (17 bits)
    reg         ifid_valid;
    reg  [15:0] ifid_instr;

    // ID/EX (48 bits)
    reg         idex_valid;
    reg  [2:0]  idex_alu_op;
    reg         idex_alu_src_imm;
    reg         idex_uses_a, idex_uses_b;
    reg         idex_reg_we, idex_mem_to_reg, idex_mem_we, idex_flag_we;
    reg         idex_jump, idex_br_z, idex_br_nz, idex_halt;
    reg  [2:0]  idex_rd, idex_ra, idex_rb;
    reg  [7:0]  idex_a_val, idex_b_val, idex_imm;

    // EX/MEM (32 bits)
    reg         exmem_valid;
    reg         exmem_reg_we, exmem_mem_to_reg, exmem_mem_we, exmem_halt;
    reg  [2:0]  exmem_rd;
    reg  [7:0]  exmem_alu_result, exmem_addr, exmem_st_data;

    // MEM/WB (14 bits)
    reg         memwb_valid;
    reg         memwb_reg_we, memwb_halt;
    reg  [2:0]  memwb_rd;
    reg  [7:0]  memwb_result;

    // ============================================================================
    // Stage 1 : IF
    // ============================================================================
    wire [15:0] if_instr;

    imem u_imem (
        .addr  (pc),
        .instr (if_instr)
    );

    // ============================================================================
    // Stage 2 : ID
    // ============================================================================
    wire [3:0] id_opcode = ifid_instr[15:12];
    wire [2:0] id_rd     = ifid_instr[11:9];
    wire [2:0] id_rs1    = ifid_instr[8:6];
    wire [2:0] id_rs2    = ifid_instr[5:3];
    wire [7:0] id_imm    = ifid_instr[7:0];

    wire [2:0] c_alu_op;
    wire       c_alu_src_imm, c_ra_is_rd, c_rb_is_rd, c_uses_a, c_uses_b;
    wire       c_reg_we, c_mem_to_reg, c_mem_we, c_flag_we;
    wire       c_jump, c_br_z, c_br_nz, c_halt;

    control_unit u_ctrl (
        .opcode      (id_opcode),
        .alu_op      (c_alu_op),
        .alu_src_imm (c_alu_src_imm),
        .ra_is_rd    (c_ra_is_rd),
        .rb_is_rd    (c_rb_is_rd),
        .uses_a      (c_uses_a),
        .uses_b      (c_uses_b),
        .reg_we      (c_reg_we),
        .mem_to_reg  (c_mem_to_reg),
        .mem_we      (c_mem_we),
        .flag_we     (c_flag_we),
        .jump        (c_jump),
        .br_z        (c_br_z),
        .br_nz       (c_br_nz),
        .halt        (c_halt)
    );

    // which register each read port reads
    wire [2:0] id_ra = c_ra_is_rd ? id_rd : id_rs1;   // ADDI reads rd on A
    wire [2:0] id_rb = c_rb_is_rd ? id_rd : id_rs2;   // ST   reads rd on B

    // write-back signals (driven in stage 5)
```

```
    wire       wb_we   = memwb_valid & memwb_reg_we & ~halted;
    wire [2:0] wb_addr = memwb_rd;
    wire [7:0] wb_data = memwb_result;

    wire [7:0] id_a_val, id_b_val;

    regfile u_rf (
        .clk     (clk),
        .rst     (rst),
        .we      (wb_we),
        .waddr   (wb_addr),
        .wdata   (wb_data),
        .ra_addr (id_ra),
        .ra_data (id_a_val),
        .rb_addr (id_rb),
        .rb_data (id_b_val)
    );

    // ============================================================================
    // Hazard unit
    // ============================================================================
    wire [1:0] fwd_a_sel, fwd_b_sel;
    wire       stall;

    hazard_unit u_hazard (
        .ifid_valid       (ifid_valid),
        .id_ra            (id_ra),
        .id_rb            (id_rb),
        .id_uses_a        (c_uses_a),
        .id_uses_b        (c_uses_b),
        .idex_valid       (idex_valid),
        .idex_ra          (idex_ra),
        .idex_rb          (idex_rb),
        .idex_uses_a      (idex_uses_a),
        .idex_uses_b      (idex_uses_b),
        .idex_rd          (idex_rd),
        .idex_reg_we      (idex_reg_we),
        .idex_mem_to_reg  (idex_mem_to_reg),
        .exmem_valid      (exmem_valid),
        .exmem_rd         (exmem_rd),
        .exmem_reg_we     (exmem_reg_we),
        .exmem_mem_to_reg (exmem_mem_to_reg),
        .memwb_valid      (memwb_valid),
        .memwb_rd         (memwb_rd),
        .memwb_reg_we     (memwb_reg_we),
        .fwd_a_sel        (fwd_a_sel),
        .fwd_b_sel        (fwd_b_sel),
        .stall            (stall)
    );

    // ============================================================================
    // Stage 3 : EX
    // ============================================================================
    // forwarding multiplexers
    reg [7:0] ex_op_a, ex_op_b;

    always @(*) begin
        case (fwd_a_sel)
            2'd1:    ex_op_a = exmem_alu_result;
            2'd2:    ex_op_a = memwb_result;
            default: ex_op_a = idex_a_val;
        endcase
        case (fwd_b_sel)
            2'd1:    ex_op_b = exmem_alu_result;
            2'd2:    ex_op_b = memwb_result;
            default: ex_op_b = idex_b_val;
        endcase
    end

    // immediate multiplexer
    wire [7:0] alu_b = idex_alu_src_imm ? idex_imm : ex_op_b;

    wire [7:0] alu_y;
    wire       alu_z, alu_c;

```

```
    alu u_alu (
        .a    (ex_op_a),
        .b    (alu_b),
        .op   (idex_alu_op),
        .c_in (c_flag),
        .y    (alu_y),
        .z    (alu_z),
        .c    (alu_c)
    );

    // branch logic (flags are always up to date in EX)
    wire take_branch = idex_valid &
                       ( idex_jump |
                        (idex_br_z  &  z_flag) |
                        (idex_br_nz & ~z_flag) );
    wire flush       = take_branch;
    wire [7:0] branch_target = idex_imm;

    // HLT anywhere from EX onward: stop fetching, squash younger instructions
    wire halt_in_pipe = (idex_valid  & idex_halt)  |
                        (exmem_valid & exmem_halt) |
                        (memwb_valid & memwb_halt);

    // ============================================================================
    // Stage 4 : MEM
    // ============================================================================
    wire [7:0] dmem_rdata;

    dmem u_dmem (
        .clk   (clk),
        .we    (exmem_valid & exmem_mem_we & ~halted),
        .addr  (exmem_addr),
        .wdata (exmem_st_data),
        .rdata (dmem_rdata)
    );

    wire [7:0] mem_result = exmem_mem_to_reg ? dmem_rdata : exmem_alu_result;

    // ============================================================================
    // Stage 5 : WB  (register write is done through u_rf above)
    // ============================================================================
    assign halted     = memwb_valid & memwb_halt;
    assign instr_done = memwb_valid & ~halt_done;
    assign pc_out     = pc;
    assign flags_out  = {z_flag, c_flag};

    // ID/EX receives a real instruction only in the normal case
    wire id_advance = ifid_valid & ~flush & ~halt_in_pipe & ~stall;

    // ============================================================================
    // The single clocked block: every pipeline register and the PC
    // ============================================================================
    always @(posedge clk) begin
        if (rst) begin
            pc               <= 8'h00;
            z_flag           <= 1'b0;
            c_flag           <= 1'b0;
            halt_done        <= 1'b0;

            ifid_valid       <= 1'b0;
            ifid_instr       <= 16'h0000;

            idex_valid       <= 1'b0;
            idex_alu_op      <= 3'd0;
            idex_alu_src_imm <= 1'b0;
            idex_uses_a      <= 1'b0;
            idex_uses_b      <= 1'b0;
            idex_reg_we      <= 1'b0;
            idex_mem_to_reg  <= 1'b0;
            idex_mem_we      <= 1'b0;
            idex_flag_we     <= 1'b0;
            idex_jump        <= 1'b0;
            idex_br_z        <= 1'b0;
            idex_br_nz       <= 1'b0;
            idex_halt        <= 1'b0;
```

```
            idex_rd          <= 3'd0;
            idex_ra          <= 3'd0;
            idex_rb          <= 3'd0;
            idex_a_val       <= 8'h00;
            idex_b_val       <= 8'h00;
            idex_imm         <= 8'h00;

            exmem_valid      <= 1'b0;
            exmem_reg_we     <= 1'b0;
            exmem_mem_to_reg <= 1'b0;
            exmem_mem_we     <= 1'b0;
            exmem_halt       <= 1'b0;
            exmem_rd         <= 3'd0;
            exmem_alu_result <= 8'h00;
            exmem_addr       <= 8'h00;
            exmem_st_data    <= 8'h00;

            memwb_valid      <= 1'b0;
            memwb_reg_we     <= 1'b0;
            memwb_halt       <= 1'b0;
            memwb_rd         <= 3'd0;
            memwb_result     <= 8'h00;
        end
        else if (halted) begin
            // ---------------- freeze: HLT is in WB ----------------
            halt_done <= 1'b1;
        end
        else begin
            // ---------------- flags (EX) ----------------
            if (idex_valid & idex_flag_we) begin
                z_flag <= alu_z;
                c_flag <= alu_c;
            end

            // ---------------- MEM/WB <- MEM ----------------
            memwb_valid  <= exmem_valid;
            memwb_reg_we <= exmem_valid & exmem_reg_we;
            memwb_halt   <= exmem_valid & exmem_halt;
            memwb_rd     <= exmem_rd;
            memwb_result <= mem_result;

            // ---------------- EX/MEM <- EX ----------------
            exmem_valid      <= idex_valid;
            exmem_reg_we     <= idex_valid & idex_reg_we;
            exmem_mem_to_reg <= idex_valid & idex_mem_to_reg;
            exmem_mem_we     <= idex_valid & idex_mem_we;
            exmem_halt       <= idex_valid & idex_halt;
            exmem_rd         <= idex_rd;
            exmem_alu_result <= alu_y;
            exmem_addr       <= ex_op_a;      // LD/ST address = rs1
            exmem_st_data    <= ex_op_b;      // ST data       = rd (port B)

            // ---------------- ID/EX <- ID ----------------
            // a bubble (id_advance = 0) has every enable forced to 0
            idex_valid       <= id_advance;
            idex_alu_op      <= c_alu_op;
            idex_alu_src_imm <= c_alu_src_imm;
            idex_uses_a      <= id_advance & c_uses_a;
            idex_uses_b      <= id_advance & c_uses_b;
            idex_reg_we      <= id_advance & c_reg_we;
            idex_mem_to_reg  <= id_advance & c_mem_to_reg;
            idex_mem_we      <= id_advance & c_mem_we;
            idex_flag_we     <= id_advance & c_flag_we;
            idex_jump        <= id_advance & c_jump;
            idex_br_z        <= id_advance & c_br_z;
            idex_br_nz       <= id_advance & c_br_nz;
            idex_halt        <= id_advance & c_halt;
            idex_rd          <= id_rd;
            idex_ra          <= id_ra;
            idex_rb          <= id_rb;
            idex_a_val       <= id_a_val;
            idex_b_val       <= id_b_val;
            idex_imm         <= id_imm;

            // ---------------- PC and IF/ID ----------------
```

```
            if (flush) begin
                pc         <= branch_target;   // go to the target
                ifid_valid <= 1'b0;            // cancel wrong-path fetch
            end
            else if (halt_in_pipe) begin
                pc         <= pc;              // fetch nothing more
                ifid_valid <= 1'b0;
            end
            else if (stall) begin
                pc         <= pc;              // same instruction waits
                ifid_valid <= ifid_valid;
                ifid_instr <= ifid_instr;
            end
            else begin
                pc         <= pc + 8'd1;
                ifid_valid <= 1'b1;
                ifid_instr <= if_instr;
            end
        end
    end

endmodule
```

A.8 sim_sources/tb_risc8_5stage.v

```
// ============================================================================
// Lê Kỳ Dũng - mã số sinh viên SE190745
// ----------------------------------------------------------------------------
// tb_risc8_5stage.v  -  self-checking testbench for risc8_5stage_core
// ----------------------------------------------------------------------------
// * 100 MHz clock (10 ns), reset held for 2 cycles
// * prints the pipeline state every cycle
// * waits for 'halted', then checks registers, memory and flags
// * prints statistics (cycles, instructions, CPI, stalls, flushes, forwards)
//   and finally *** ALL TESTS PASSED *** or *** TEST FAILED ***
// ============================================================================
`timescale 1ns / 1ps

module tb_risc8_5stage;

    reg        clk = 1'b0;
    reg        rst = 1'b1;
    wire       halted;
    wire [7:0] pc_out;
    wire [1:0] flags_out;
    wire       instr_done;

    risc8_5stage_core dut (
        .clk        (clk),
        .rst        (rst),
        .halted     (halted),
        .pc_out     (pc_out),
        .flags_out  (flags_out),
        .instr_done (instr_done)
    );

    // ---------------- clock: 100 MHz ----------------
    always #5 clk = ~clk;

    // ---------------- statistics ----------------
    integer cycle       = 0;
    integer n_instr     = 0;
    integer n_stall     = 0;
    integer n_flush     = 0;
    integer n_fwd       = 0;
    integer errors      = 0;

    always @(posedge clk) begin
        if (!rst && !halted) begin
            cycle = cycle + 1;
            if (instr_done)              n_instr = n_instr + 1;
            if (dut.stall && !dut.flush) n_stall = n_stall + 1;
```

```
            if (dut.flush)               n_flush = n_flush + 1;
            if (dut.fwd_a_sel != 2'd0)   n_fwd   = n_fwd + 1;
            if (dut.fwd_b_sel != 2'd0)   n_fwd   = n_fwd + 1;

            $display("cyc %3d | PC=%h | IF/ID %s %h | ID/EX %b EX/MEM %b MEM/WB %b | stall=%b flush=%b fwdA
 =%0d fwdB=%0d | Z=%b C=%b",
                     cycle, dut.pc,
                     dut.ifid_valid ? "v" : "-", dut.ifid_instr,
                     dut.idex_valid, dut.exmem_valid, dut.memwb_valid,
                     dut.stall, dut.flush, dut.fwd_a_sel, dut.fwd_b_sel,
                     dut.z_flag, dut.c_flag);
        end
    end

    // ---------------- check helper ----------------
    task check;
        input [8*16-1:0] name;
        input [7:0]      got;
        input [7:0]      exp;
        begin
            if (got === exp)
                $display("  PASS  %0s = 0x%h", name, got);
            else begin
                $display("  FAIL  %0s = 0x%h (expected 0x%h)", name, got, exp);
                errors = errors + 1;
            end
        end
    endtask

    // ---------------- main sequence ----------------
    initial begin
        $display("==== risc8 5-stage pipeline simulation ====");
        rst = 1'b1;
        repeat (2) @(posedge clk);
        #1 rst = 1'b0;

        // wait for HLT to reach WB (with a timeout)
        fork : wait_halt
            begin
                wait (halted === 1'b1);
                disable wait_halt;
            end
            begin
                repeat (2000) @(posedge clk);
                $display("*** TIMEOUT: processor did not halt ***");
                $finish;
            end
        join

        // HLT in WB is the last instruction completing
        cycle   = cycle + 1;
        n_instr = n_instr + 1;
        @(posedge clk); #1;

        $display("");
        $display("==== Results ====");
        check("R1",        dut.u_rf.regs[1],     8'h00);
        check("R2",        dut.u_rf.regs[2],     8'h0F);
        check("R3",        dut.u_rf.regs[3],     8'h01);
        check("R4",        dut.u_rf.regs[4],     8'h20);
        check("R5",        dut.u_rf.regs[5],     8'h00);
        check("R6",        dut.u_rf.regs[6],     8'h66);
        check("R7",        dut.u_rf.regs[7],     8'h00);
        check("MEM[0x20]", dut.u_dmem.mem[8'h20], 8'h0F);
        check("Z,C",       {6'b0, flags_out},    8'h03);

        $display("");
        $display("==== Statistics ====");
        $display("  cycles            : %0d", cycle);
        $display("  instructions      : %0d", n_instr);
        $display("  CPI               : %0d.%02d", cycle / n_instr, (cycle * 100 / n_instr) % 100);
        $display("  load-use stalls   : %0d", n_stall);
        $display("  taken branches    : %0d (2-cycle flush each)", n_flush);
        $display("  forwarded operands: %0d", n_fwd);
        $display("");
```

```
122
123         if (errors == 0)
124             $display("*** ALL TESTS PASSED ***");
125         else
126             $display("*** TEST FAILED: %0d error(s) ***", errors);
127         $finish;
128     end
129
130 endmodule
```

A.9 constraints/risc8.xdc

```
1  ## ============================================================================
2  ## Lê Kỳ Dũng - mã số sinh viên SE190745
3  ## ----------------------------------------------------------------------------
4  ## risc8.xdc  -  constraints for risc8_5stage_core
5  ## ----------------------------------------------------------------------------
6  ## Tells Vivado the target clock: 100 MHz (10 ns period).
7  ## Used by synthesis, implementation and the timing check (WNS must be >= 0).
8  ## No pin assignments: this project is simulated and timed only, not loaded
9  ## onto a real board.
10 ## ============================================================================
11
12 create_clock -period 10.000 -name sys_clk -waveform {0.000 5.000} [get_ports clk]
```

B Reproducing the Results in Vivado

1. Create a new RTL project in Vivado 2026.1 and select the part `xc7a35tcpg236-1`.
2. Add the seven files of Appendix A as design sources, `risc8.xdc` as a constraint file and `tb_risc8_5stage.v` as a simulation source. Set `risc8_5stage_core` as the design top and `tb_risc8_5stage` as the simulation top.
3. Run **Run Simulation** → **Run Behavioral Simulation**. The Tcl Console must end with `*** ALL TESTS PASSED ***` at 526 ns.
4. Run **Run Synthesis**, then **Run Implementation**.
5. Open the implemented design and run **Report Timing Summary** and **Report Utilization**. WNS must be positive.